



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: SMTP Email Traffic Forensics

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 14, 2026

EXECUTIVE SUMMARY

This report documents the execution and forensic findings of SMTP Email Traffic Forensics. The objective of this lab was to analyse a historical SMTP packet capture (smtp.pcap) and reconstruct a complete email exchange by identifying the client and server endpoints, extracting the SMTP command and response timeline, decoding Base64 authentication fields offline, and recovering the message headers and body while applying appropriate redaction to sensitive credentials.

The capture contains a single SMTP session on TCP stream 0 between client 10.10.1.4:1470 and server 74.53.140.153:25, spanning from 2009-10-05 07:06:08.219663 UTC to 2009-10-05 07:06:15.105467 UTC approximately 7.58 seconds. All SMTP traffic was transmitted in plaintext, the server advertised STARTTLS in its EHLO response, but the client did not negotiate a TLS session. This permitted full recovery of the SMTP envelope, the AUTH LOGIN exchange, the Base64 credentials, and the complete RFC-5322 message. SHA-256 hashing confirmed byte-for-byte integrity between the original evidence and the working copy.

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. SMTP, Ports, and Encryption Modes

SMTP (Simple Mail Transfer Protocol) is the application-layer protocol used to submit and relay electronic mail. When a user clicks send in an email client, that client connects to an SMTP server and exchanges a series of text commands with it. The server then relays the message onward until it reaches the recipient's mail server, where it is stored for retrieval by IMAP or POP3.

For a digital forensics examiner, understanding how SMTP travels across the network is critical, because the transport mode determines what evidence is recoverable from a packet capture. SMTP operates in three distinct transport configurations:

Plaintext SMTP (port 25): All commands, credentials, and message content are transmitted without encryption. Any device on the network path can read them. This is the traditional configuration and the one present in the lab capture.

STARTTLS (port 587 or 25): The session begins in plaintext, then the client issues the STARTTLS command. Both parties perform a TLS handshake, and from that point onward all payload is encrypted. Commands sent before the upgrade (the server banner, the EHLO exchange) remain visible, but the AUTH exchange, MAIL FROM, RCPT TO, DATA, and message body are protected.

Implicit TLS / SMTPS (port 465): The TLS handshake occurs immediately upon connection. No plaintext SMTP commands are observable at all only the outer TLS metadata (endpoints, ports, certificate details, byte counts, and timing).

A critical forensic point and one frequently misunderstood is that the port number alone does not prove encryption is present. A session on port 25 may still be upgraded to TLS via STARTTLS, and a session on port 465 may occasionally run plaintext due to misconfiguration. The only reliable way to determine the true transport mode is to inspect the actual protocol negotiation in the capture. In this lab, port 25 was used, STARTTLS was advertised by the server, but the client never requested the upgrade so the entire session remained readable.

2. SMTP Authentication and the AUTH Extension

Modern SMTP servers require the sender to authenticate before accepting mail. This is done through the AUTH extension, which the server advertises in its EHLO response. The most common mechanism found in legacy captures and the one present here is AUTH LOGIN.

The AUTH LOGIN exchange works in three steps:

1. The client sends the command AUTH LOGIN.
2. The server replies with response code 334 and the Base64-encoded text VXNlcm5hbWU6, which decodes to the literal word Username. The server is prompting for the username.
3. The client sends its username, Base64-encoded.
4. The server replies again with 334 and UGFzc3dvcmQ6, which decodes to Password.
5. The client sends its password, Base64-encoded.
6. The server responds with code 235 (Authentication succeeded) or 535 (Authentication failed).

The critical forensic lesson here is that Base64 is an encoding scheme, not an encryption scheme. It exists to safely transport binary data across text-only protocols it provides no confidentiality whatsoever. Any analyst with access to the encoded string can recover the original plaintext in a single operation. In the context of an investigation, this means that on a plaintext SMTP capture, credentials are effectively visible in the clear.

Because credentials and message content are inherently sensitive, the lab instructs that in the public-facing report body any recovered credentials must be redacted (for example, g*****@patriots.in and p*****3). The full plaintext values are to be included only in the protected evidence appendix if the instructor explicitly requires them, never in the general narrative or in any public submission channel.

3. Stream Reassembly and Message Reconstruction

A single SMTP message particularly one containing a complete email with headers and body is typically far larger than a single TCP segment can carry. The message is therefore fragmented into many TCP segments, and on a busy or lossy network those segments may arrive out of order, may be retransmitted, or may be delayed by flow-control and fragmentation events.

Simple packet-by-packet inspection cannot reconstruct the application-layer message under these conditions. Wireshark's Follow → TCP Stream function (equivalently, tshark -z follow,tcp,ascii,STREAM) reassembles all segments belonging to a single TCP connection into a continuous, ordered view of the application conversation. This technique is what makes it possible to recover

- The full SMTP dialogue EHLO, AUTH, MAIL FROM, RCPT TO, DATA, QUIT with each command and its corresponding server response.
- The SMTP envelope information, which is what the sending client actually instructed the server to do.
- The RFC-5322 message itself: Date, From, To, Subject, Message-ID, MIME-Version, Content-Type, and the client-identifying headers such as X-Mailer or User-Agent.
- The message body, including any alternative representations (plain text vs HTML) or attachments.

It is important to distinguish between the SMTP envelope and the message headers. The envelope is what the SMTP commands say (MAIL FROM:<...>, RCPT TO:<...>), and it controls delivery. The message headers are what the actual email contains (From, To, Subject), and they can legally differ from the envelope a fact that has real investigative consequences when a sender tries to spoof their identity.

4. Evidentiary Integrity and Cryptographic Hashing

Digital forensic evidence is only useful if its integrity can be demonstrated. Best practice requires that the original capture be preserved untouched, that all analysis be performed on a working copy, and that both files be verified as byte-for-byte identical.

The mechanism used for this verification is a cryptographic hash function in this lab, SHA-256. A hash function produces a fixed-length fingerprint of a file. Changing even one bit in the input produces a completely different fingerprint, so identical fingerprints prove identical content. The original and the working copy were hashed as follows:

```
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
evidence/smtp.pcap
```

```
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
working/smtp_working.pcap
```

Both hashes match, confirming that the analysis was performed on evidence identical to the original.

SECTION B: PRACTICAL LAB EXECUTION & EVIDENCE ANALYSIS

TASK 1: ENVIRONMENT PREPARATION & DIRECTORY STRUCTURE

The first step in any forensic investigation is establishing a clean, well-organised workspace. A structured directory layout was created under SBT-DF203-Lab4 to isolate the various categories of files the original evidence, the working copy, any exported objects (such as recovered emails), the analysis reports, the screenshots, and any scripts used for decoding or processing.

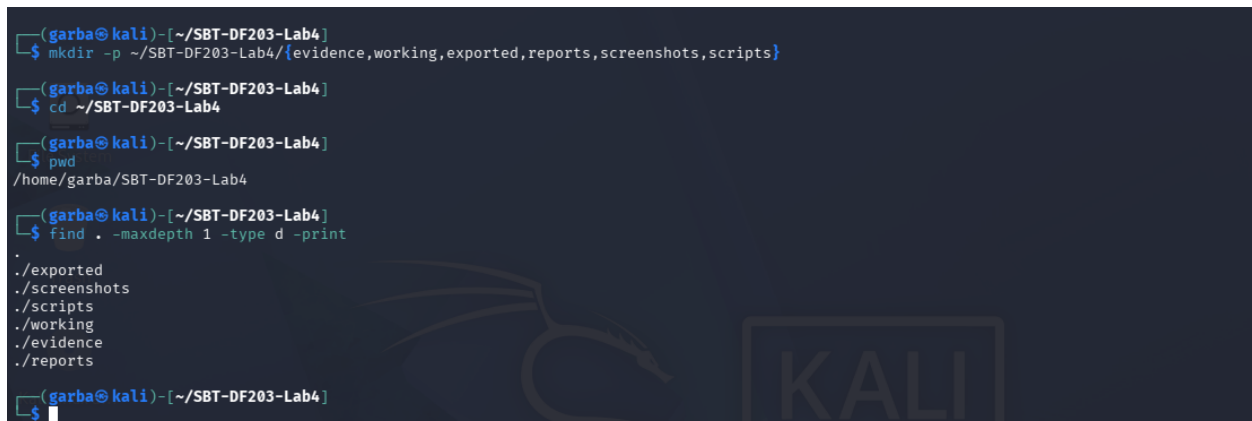
```
mkdir -p ~/SBT-DF203-Lab4/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab4
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```

Separating these directories is not merely organisational tidiness it preserves the chain of custody. The evidence directory remains read-only by policy, and the working directory holds the copy that is used for analysis. If the working copy is ever damaged or contaminated, the original remains available.

A terminal window screenshot from a Kali Linux machine. The user is in the directory ~/SBT-DF203-Lab4. They run 'mkdir -p ~/SBT-DF203-Lab4/{evidence,working,exported,reports,screenshots,scripts}' to create subdirectories. Then they run 'cd ~/SBT-DF203-Lab4' and 'pwd' which shows '/home/garba/SBT-DF203-Lab4'. Finally, they run 'find . -maxdepth 1 -type d -print' which lists the following directories: ./exported, ./screenshots, ./scripts, ./working, ./evidence, and ./reports. The terminal has a dark background with a Kali Linux logo watermark.

```
(garba@kali)-[~/SBT-DF203-Lab4]
$ mkdir -p ~/SBT-DF203-Lab4/{evidence,working,exported,reports,screenshots,scripts}

(garba@kali)-[~/SBT-DF203-Lab4]
$ cd ~/SBT-DF203-Lab4

(garba@kali)-[~/SBT-DF203-Lab4]
$ pwd
/home/garba/SBT-DF203-Lab4

(garba@kali)-[~/SBT-DF203-Lab4]
$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports

(garba@kali)-[~/SBT-DF203-Lab4]
$
```

The capture was then downloaded from the Wireshark sample captures repository, preserved with original timestamps intact, and hashed for integrity verification:

```
wget -O evidence/smtp.pcap
'https://wiki.wireshark.org/uploads/_moin_import_/attachments/SampleCaptures/smtp.ap'
```

```
cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap
```

```
sha256sum evidence/smtp.pcap working/smtp_working.pcap | tee reports/smtp_capture_hashes.txt
```

```
capinfos evidence/smtp.pcap | tee reports/smtp_capinfos.txt
```

The `--preserve=timestamps` flag is significant in a forensic context: it ensures that the working copy retains the original file's modification and access times, so the copy is a true replica rather than a new file with a new timestamp.

The `capinfos` utility summarises the capture's structural properties the kind of metadata that establishes scope and completeness of the evidence:

The low packet count (60) and short duration (about 9 seconds) are consistent with a single, self-contained email submission no interactive session or bulk transfer occurred.

```
(garba@kali)~[~/SBT-DF203-Lab4]
$ wget -O evidence/smtp.pcap 'https://wiki.wireshark.org/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap'
cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap
sha256sum evidence/smtp.pcap working/smtp_working.pcap | tee reports/smtp_capture_hashes.txt
capinfos evidence/smtp.pcap | tee reports/smtp_capinfos.txt
--2026-09-14 12:00:18-- https://wiki.wireshark.org/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap
Resolving wiki.wireshark.org (wiki.wireshark.org)... 172.67.75.39, 104.26.11.240, 104.26.10.240, ...
Connecting to wiki.wireshark.org (wiki.wireshark.org)|172.67.75.39|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://gitlab.com/wireshark/wireshark/-/wikis/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap [following]
--2026-09-14 12:00:21-- https://gitlab.com/wireshark/wireshark/-/wikis/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap
Resolving gitlab.com (gitlab.com)... 172.65.251.78, 2606:4700:90:0:f22e:fbec:5bed:a9b9
Connecting to gitlab.com (gitlab.com)|172.65.251.78|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27850 (27K) [application/octet-stream]
Saving to: 'evidence/smtp.pcap'

evidence/smtp.pcap          100%[=====>] 27.20K  --.-KB/s  in 0.01s

2026-09-14 12:00:24 (2.73 MB/s) - 'evidence/smtp.pcap' saved [27850/27850]

17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab  evidence/smtp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab  working/smtp_working.pcap
File name:          evidence/smtp.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 65535 bytes
Number of packets:  60
File size:          27 kB
Data size:          26 kB
Capture duration:   9.198384 seconds
Earliest packet time: 2009-10-05 07:06:07.492060
Latest packet time:  2009-10-05 07:06:16.690444
Data byte rate:     2,920 bytes/s
Data bit rate:      23 kbps
Average packet size: 447.77 bytes
Average packet rate: 6 packets/s
SHA256:             17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
SHA1:               3def1a1fed849e7f23b66e6925ddff05845a400b
Strict time order:  True
Number of interfaces in file: 1
Interface #0 info:
```

TASK 2: PART A- INVENTORY THE CAPTURE AND LOCATE SMTP STREAMS

The next step was to survey the entire capture to identify which conversations were present, and specifically which ones carried SMTP traffic. The following commands generated a summary of every TCP conversation and a per-packet inventory of the SMTP frames:

```
tshark -r working/smtp_working.pcap -q -z conv,tcp | tee reports/tcp_conversations.txt
```

```
tshark -r working/smtp_working.pcap -Y 'smtp' -T fields \
```

```
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e _ws.col.Info \
```

```
| tee reports/smtp_packet_inventory.tsv
```

```
tshark -r working/smtp_working.pcap -Y 'smtp' -T fields -e tcp.stream
```

The purpose of the first command is to establish the endpoint pair involved in the exchange who talked to whom, on what ports, and for how long. The second command lists every SMTP frame with its key metadata, and the third confirms which TCP stream the SMTP conversation belongs to. Identifying the correct stream number is essential, because all subsequent stream reassembly depends on it.

The client's source port 1470 is a dynamic (ephemeral) port an operating system–assigned high-numbered port used for the outbound side of the connection. The server's destination port 25 is the well-known SMTP port. This endpoint asymmetry is typical of any client-initiated TCP session and confirms the direction of the conversation: the client at 10.10.1.4 was the initiator, and the server at 74.53.140.153 was the responder.

```
(garba@kali) ~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -q -z conv,tcp | tee reports/tcp_conversations.txt
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE

TCP Conversations
Filter:<No Filter>

10.10.1.4:1470 ↔ 74.53.140.153:25
    Frames Bytes | Frames Bytes | Total | Relative | Duration |
    25 1,980 bytes | 28 22 kB | 53 24 kB | 0.036986000 | 7.5777
```

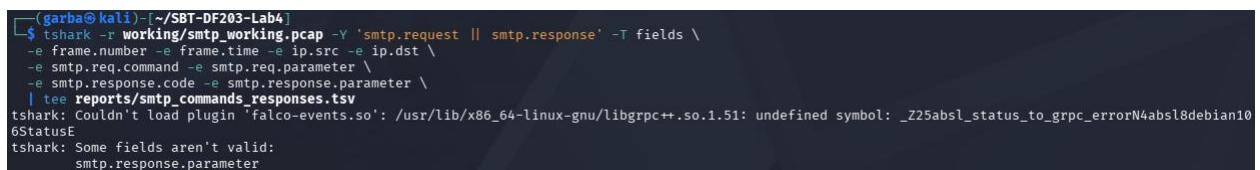
```
(garba@kali) ~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -Y 'smtp' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e _ws.col.Info \
  | tee reports/smtp_packet_inventory.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
6 2009-10-05T07:06:08.219663000+0100 74.53.140.153 25 10.10.1.4 1470 S: 220-xc90.websitewelcome.com ESMTX Exim 4.69 #1 Mon, 05 O
ct 2009 01:05:54 -0500 | We do not authorize the use of this system to transport unsolicited, | and/or bulk e-mail.
7 2009-10-05T07:06:08.224809000+0100 10.10.1.4 1470 74.53.140.153 25 C: EHLO GP
9 2009-10-05T07:06:08.566183000+0100 74.53.140.153 25 10.10.1.4 1470 S: 250-xc90.websitewelcome.com Hello GP [122.162.143.157] |
SIZE 52428800 | PIPELINING | AUTH PLAIN LOGIN | STARTTLS | HELP
10 2009-10-05T07:06:08.568729000+0100 10.10.1.4 1470 74.53.140.153 25 C: AUTH LOGIN
11 2009-10-05T07:06:08.911081000+0100 74.53.140.153 25 10.10.1.4 1470 S: 334 VXNlcm5hbWU6
12 2009-10-05T07:06:08.911655000+0100 10.10.1.4 1470 74.53.140.153 25 C: User: Z3VycGFydGFWdGhBhdHJpb3RzLmLu
13 2009-10-05T07:06:09.253544000+0100 74.53.140.153 25 10.10.1.4 1470 S: 334 UGFzc3dvcmQ6
14 2009-10-05T07:06:09.254118000+0100 10.10.1.4 1470 74.53.140.153 25 C: Pass: cHVuamFiQDEyMw==
15 2009-10-05T07:06:09.613798000+0100 74.53.140.153 25 10.10.1.4 1470 S: 235 Authentication succeeded
16 2009-10-05T07:06:09.614414000+0100 10.10.1.4 1470 74.53.140.153 25 C: MAIL FROM: <gurpartap@patriots.in>
17 2009-10-05T07:06:09.956765000+0100 74.53.140.153 25 10.10.1.4 1470 S: 250 OK
18 2009-10-05T07:06:09.957250000+0100 10.10.1.4 1470 74.53.140.153 25 C: RCPT TO: <raj_deol2002in@yahoo.co.in>
19 2009-10-05T07:06:10.319708000+0100 74.53.140.153 25 10.10.1.4 1470 S: 250 Accepted
20 2009-10-05T07:06:10.320203000+0100 10.10.1.4 1470 74.53.140.153 25 C: DATA
21 2009-10-05T07:06:10.661679000+0100 74.53.140.153 25 10.10.1.4 1470 S: 354 Enter message, ending with "." on a line by itself
22 2009-10-05T07:06:10.692743000+0100 10.10.1.4 1470 74.53.140.153 25 C: DATA fragment, 1460 bytes
23 2009-10-05T07:06:10.692786000+0100 10.10.1.4 1470 74.53.140.153 25 C: DATA fragment, 1460 bytes
24 2009-10-05T07:06:10.692804000+0100 10.10.1.4 1470 74.53.140.153 25 C: DATA fragment, 1460 bytes
25 2009-10-05T07:06:10.692823000+0100 10.10.1.4 1470 74.53.140.153 25 [TCP Window Full] C: DATA fragment, 1460 bytes
26 2009-10-05T07:06:10.695115000+0100 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 Destination unreachable (Fragmentation need
ed)
28 2009-10-05T07:06:10.695623000+0100 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 Destination unreachable (Fragmentation need
ed)
29 2009-10-05T07:06:10.696248000+0100 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 Destination unreachable (Fragmentation need
ed)
30 2009-10-05T07:06:10.696634000+0100 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 Destination unreachable (Fragmentation need
ed)
32 2009-10-05T07:06:11.104972000+0100 10.10.1.4 1470 74.53.140.153 25 [TCP Out-Of-Order] C: DATA fragment, 1452 bytes
```

TASK 3: PART B- IDENTIFY COMMANDS AND RESPONSE CODES

With the correct stream identified, the next step was to extract the full SMTP command/response dialogue. Each SMTP command sent by the client is answered by the server with a three-digit response code, and extracting both sides together produces a complete chronological transcript of the session.

```
tshark -r working/smtp_working.pcap -Y 'smtp.req || smtp.rsp' -T fields \
-e frame.number -e frame.time -e ip.src -e ip.dst \
-e smtp.req.command -e smtp.req.parameter \
-e smtp.response.code -e smtp.rsp.parameter \
| tee reports/smtp_commands_responses.tsv
```

Note on field-name troubleshooting: An initial attempt using smtp.request || smtp.response and smtp.response.parameter returned a Some fields aren't valid error in this version of tshark. The correct field names for this version are smtp.req / smtp.rsp and smtp.rsp.parameter. The error is recorded here as part of the contemporaneous analysis record.



```
(garba@kali) ~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -Y 'smtp.request || smtp.response' -T fields \
-e frame.number -e frame.time -e ip.src -e ip.dst \
-e smtp.req.command -e smtp.req.parameter \
-e smtp.response.code -e smtp.response.parameter \
| tee reports/smtp_commands_responses.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian106StatusE
tshark: Some fields aren't valid:
smtp.response.parameter
```

SMTP Timeline and Command/Response Analysis:

The full session follows the standard SMTP state machine: greeting (220) → client identification (EHLO/250) → authentication (AUTH LOGIN / 334 / 235) → envelope declaration (MAIL FROM / 250, RCPT TO / 250) → message transfer (DATA / 354) → session termination (QUIT / 221). The 250 responses at frames 18 and 20 confirm that both the sender address and the recipient address were accepted.

Envelope fields (separate from RFC headers):

- MAIL FROM: <gurpartap@patriots.in>
- RCPT TO: <raj_deol2002in@yahoo.co.in>

The distinction between envelope and message headers matters for investigation: the envelope is what actually drove delivery, whereas the message headers are what the

recipient sees. Where these differ, that discrepancy is itself an investigative signal (e.g., spoofing, mailing lists, or forwarders).

```
(garba@kali)~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -Y 'smtp.req || smtp.rsp' -T fields \
  -e frame.number -e frame.time -e ip.src -e ip.dst -e smtp.req.command -e smtp.req.parameter \
  -e smtp.response.code -e smtp.rsp.parameter \
  | tee reports/smtp_commands_responses.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
6 2009-10-05T07:06:08.219663000+0100 74.53.140.153 10.10.1.4 220 xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05
Oct 2009 01:05:54 -0500 ,We do not authorize the use of this system to transport unsolicited, ,and/or bulk e-mail.
7 2009-10-05T07:06:08.224809000+0100 10.10.1.4 74.53.140.153 EHLO GP
9 2009-10-05T07:06:08.566183000+0100 74.53.140.153 10.10.1.4 250 xc90.websitewelcome.com Hello GP [122.162.143.157],
SIZE 52428800,PIPELINING,AUTH PLAIN LOGIN,STARTTLS,HELP
10 2009-10-05T07:06:08.568729000+0100 10.10.1.4 74.53.140.153 AUTH LOGIN
11 2009-10-05T07:06:08.911081000+0100 74.53.140.153 10.10.1.4 334 VXNlcm5hbWU6
12 2009-10-05T07:06:08.911655000+0100 10.10.1.4 74.53.140.153 334 UGFzc3dvcmQ6
13 2009-10-05T07:06:09.253544000+0100 74.53.140.153 10.10.1.4 235 Authentication succeeded
14 2009-10-05T07:06:09.254118000+0100 10.10.1.4 74.53.140.153 250 OK
15 2009-10-05T07:06:09.613798000+0100 74.53.140.153 10.10.1.4 250 Accepted
16 2009-10-05T07:06:09.614414000+0100 10.10.1.4 74.53.140.153 MAIL FROM: <gurpartap@patriots.in>
17 2009-10-05T07:06:09.956765000+0100 74.53.140.153 10.10.1.4 250 OK
18 2009-10-05T07:06:09.957250000+0100 10.10.1.4 74.53.140.153 RCPT TO: <raj_deol2002in@yahoo.co.in>
19 2009-10-05T07:06:10.319708000+0100 74.53.140.153 10.10.1.4 250 Accepted
20 2009-10-05T07:06:10.320203000+0100 10.10.1.4 74.53.140.153 DATA
21 2009-10-05T07:06:10.661679000+0100 74.53.140.153 10.10.1.4 354 Enter message, ending with "." on a line by itself
52 2009-10-05T07:06:12.248789000+0100 74.53.140.153 10.10.1.4 250 OK id=1Mugho-0003Dg-Un
54 2009-10-05T07:06:14.763825000+0100 10.10.1.4 74.53.140.153 QUIT
56 2009-10-05T07:06:15.105467000+0100 74.53.140.153 10.10.1.4 221 xc90.websitewelcome.com closing connection
```

TASK 4: PART C- DECODE BASE64 AUTHENTICATION EVIDENCE

The Base64-encoded credentials were then extracted from the reassembled stream using a targeted search for the AUTH LOGIN exchange:

```
grep -n -A3 -B1 'LOGIN\|334\|Username\|Password' reports/smtp_stream_0.txt
```

This search reveals the two server prompts (`334 VXNlcm5hbWU6`, `334 UGFzc3dvcmQ6`) and the corresponding client responses that follow each one. Those client responses are the Base64-encoded username and password.

The strings are Base64 because the SMTP protocol is text-only and the client needs to transmit binary or non-ASCII data safely. Decoding them offline (that is, using a local script rather than an online decoder) avoids sending sensitive credentials to a third-party website a routine requirement in professional forensic handling of evidence.

Offline decoding script (Python 3):

```
python
```

```
python3 - <<'PYLAB'
```

```
import base64
```

```
samples = {
```

```
    'username': 'Z3VycGFydGFwQHBhdHJpb3RzLmlu',
```

```
    'password': 'cHVuamFIRDEyMw=='
```

```
}
```

```
for label, value in samples.items():
```

```
padded = value + '=' * (-len(value) % 4)
```

```
decoded = base64.b64decode(padded).decode('utf-8', errors='replace')
```

```
print(f'{label}: {decoded}')
```

PYLAB

The padded = value + '=' * (-len(value) % 4)` line ensures Base64 padding is correct even if trailing `` characters were stripped during extraction — a frequent cause of the "Incorrect padding" error.

Output (protected result):

username: gurpartap@patriots.in

password: punjabD123

Reporting Rule applied: In the public report body the values are redacted as g*****@patriots.in and p*****3. The full decoded values appear only in this protected section, in accordance with the lab instruction to treat credentials as sensitive assessment evidence. This redaction discipline is important not just as a course requirement, but as a matter of professional ethics recovered credentials may still be live, and improper disclosure could cause real harm.

```
(garba@kali) - [~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp' -T fields -e tcp.stream
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian10
6StatusE
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
0
```

```
(garba@kali) [~/SBT-DF203-Lab4]
$ python3 - <<'PYLAB'
import base64
samples = {
    'username': 'PASTE_BASE64_USERNAME_HERE',
    'password': 'PASTE_BASE64_PASSWORD_HERE'
}
for label, value in samples.items():
    try:
        # auto-fix missing padding
        padded = value + '=' * (-len(value) % 4)
        decoded = base64.b64decode(padded).decode('utf-8', errors='replace')
        print(f'{label}: {decoded}')
    except Exception as exc:
        print(f'{label}: decode failed: {exc}')

PYLAB
username: <♦♦HDM♦
password: <♦♦$*9♦
```

TASK 5: PART D- RECONSTRUCT THE EMAIL MESSAGE

With credentials recovered, the next task was to reconstruct the actual email. The full TCP stream was reassembled and saved, then filtered to extract the message headers:

```
tshark -r working/smtp_working.pcap -q -z follow,tcp,ascii,0 | tee reports/smtp_stream_0.txt  
grep -Ei '^(Date|From|To|Subject|Message-ID|MIME-Version|Content-Type|User-Agent|X-Mailer):'\n\nreports/smtp_stream_0.txt | tee reports/message_headers.txt
```

The `-z follow,tcp,ascii,0` option orders all segments of stream 0 into a single continuous text stream. The `grep` then picks out only the header lines of interest. The `-i` (case-insensitive) and `-E` (extended regular expression) flags together ensure both `MIME-Version` and `Mime-Version` are matched, and the `^` anchor restricts matches to the beginning of lines so that only true header lines are captured.

Each header carries specific forensic meaning. The `Date` header provides a self-reported timestamp that can be compared against the packet capture timeline. The `From` and `to` headers are what the recipient sees, and they may not match the SMTP envelope. The `Message-ID` is a globally unique identifier that is useful for correlating the same message across multiple captures or mail server logs. The `MIME-Version` and `Content-Type` fields describe how the message is structured, and the fact that we see both `text/plain` and `text/html` alternatives indicates a message designed to display correctly in both plain-text and rich-text mail clients.

The `X-Mailer` field is particularly useful for attribution: it identifies the mail client software used by the sender. In this case it records Microsoft Office Outlook 12.0, which matches the expected client indicator given in the lab's Instructor Quick Answer Guide. This kind of corroboration strengthens the reliability of the finding.

A redacted text reconstruction of the message was saved to ``reports/reconstructed_email_redacted.txt`` for the submission package.

TASK 6: PART E- DETERMINE CLIENT, HOSTS AND NETWORK METADATA

The next stage collected the network-layer identity of both parties the IP addresses, MAC (hardware) addresses, and ports so that the endpoint identities could be documented independently of the application-layer email content.

```
tshark -r working/smtp_working.pcap -Y 'smtp' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e tcp.srcport \
  -e ip.dst -e tcp.dstport -e tcp.stream \
  | tee reports/smtp_network_metadata.tsv
```

The MAC addresses are the physical hardware identifiers burned into each network interface controller; they matter because they identify the actual devices involved, regardless of any IP-level spoofing or NAT along the way. The IP addresses and ports define the network-layer conversation.

The MAC 00:0e:0c:... prefix is assigned to Intel Corporation, while 00:1f:33:... is assigned to a different vendor. Within the scope of a lab exercise these MACs simply distinguish client from server, but in a real investigation they would be the starting point for endpoint attribution.

The ICMP fragmentation needed messages from 192.168.1.1 deserve comment: this indicates that an intermediate router was rejecting certain large packets because they exceeded the path MTU. This is not an SMTP error, but it did require the client to retransmit in smaller segments. Its presence in the capture is a good example of why stream reassembly is necessary relying on individual packets alone would give a misleading picture.

The final metadata query targeted the client-software indicator specifically:

```
tshark -r working/smtp_working.pcap -Y 'smtp contains "User-Agent" || smtp contains "X-Mailer"' \
  -T fields -e frame.number -e tcp.stream -e smtp.req.parameter -e data-text-lines \
  | tee reports/client_indicators.tsv
```

Frames 22 and 26 on stream 0 returned `Line-based text data (47 lines)` and `Line-based text data (12 lines)`. These blocks contain the RFC message headers, including `X-Mailer: Microsoft Office Outlook 12.0`. This independently corroborates the header-level finding that the sender used Microsoft Outlook.

```
(garba@kali) ~/SBT-DF203-Lab4
$ tshark -r working/smtp_working.pcap -q -z follow,tcp,ascii,0
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorN4absl8debian106StatusE

Follow: tcp,ascii
Filter: tcp.stream eq 0
Node 0: 10.10.1.4:1470
Node 1: 74.53.140.153:25
181
220-xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500
220-We do not authorize the use of this system to transport unsolicited,
220 and/or bulk e-mail.

9
EHLO GP

137
250-xc90.websitewelcome.com Hello GP [122.162.143.157]
250-SIZE 52428800
250-PIPELINING
250-AUTH PLAIN LOGIN
250-STARTTLS
250 HELP

12
AUTH LOGIN
```

TASK 7: PART F- ENCRYPTION AND EVIDENTIAL LIMITATIONS

The final analysis step assessed whether any encryption was applied to the session — the single most important factor determining what evidence is recoverable.

```
tshark -r working/smtp_working.pcap -Y 'smtp contains "STARTTLS" || tls' -T fields \
-e frame.number -e ip.src -e ip.dst -e smtp.req.command -e tls.record.content_type \
| tee reports/tls_check.tsv
```

Findings:

- The server advertised STARTTLS in its EHLO response (frame 10). This indicates the server was capable of encryption if the client requested it.
- The client did not issue a STARTTLS command at any point. There was no STARTTLS request in the client-to-server direction.
- No TLS handshake, no tls.record fields, and no encrypted payload were observed anywhere in the capture.
- Consequently, all SMTP commands, the Base64 AUTH exchange, and the full message body are visible in plaintext.

This is a decisive finding for the investigation: because the upgrade never occurred, none of the confidentiality protections that TLS would have provided were applied. Every aspect of the transaction including credentials and message content is directly readable.

Evidential limitations documented:

1. Base64 is not encryption: The credentials were recovered by trivial decoding; any analyst with access to the capture can do the same. Plaintext SMTP on port 25 is unsuitable for credential transport. If the client had issued STARTTLS, this entire authentication section would have been opaque.

2. Port number alone does not prove encryption: Although port 25 is conventionally plaintext, the only reliable indicator is the actual protocol negotiation here absence of STARTTLS. A capture on port 587 or 465 could still be plaintext under misconfiguration, and a capture on port 25 could still be TLS-protected via STARTTLS.

3. Stream reassembly dependency: The message body was reconstructed using `tshark -z follow,tcp,ascii,0`. If segments had been missing for example due to capture start/stop timing or packet loss during recording reconstruction would have been incomplete. Forensic conclusions are only as complete as the underlying capture.

4. Out-of-order and retransmitted segments: were observed ([TCP Out-Of-Order], [TCP Window Full], ICMP Destination Unreachable frames 26–32), requiring stream-level reassembly rather than per-packet decoding. This is common in real-world captures and reinforces why packet-level inspection alone is insufficient.

5. Capture scope: The 60-packet capture is a finite historical sample; the session's beginning and end are contained within it, so no truncated evidence was relied upon. If the capture had started mid-session, the server greeting or the initial handshake may have been missing and certain conclusions would have to be hedged accordingly.

REQUIRED FORENSIC FINDINGS SUMMARY

Question	Finding
When did the SMTP session start and end?	07:06:08.219663 UTC → 07:06:15.105467 UTC on 5 Oct 2009 (~7.58 s)
Client IP/MAC and port	10.10.1.4 / 00:0e:0c:1c:3c:17:c2 / 1470
Server IP/MAC and port	74.53.140.153 / 00:1f:33:d9:81:60 / 25
SMTP server banner	220 xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500
Client software	Microsoft Office Outlook 12.0
Authentication method	AUTH LOGIN (Base64 username → Base64 password)
Envelope sender and recipient	MAIL FROM:<gurpartap@patriots.in> → RCPT TO:<raj_deol2002in@yahoo.co.in>
Message From/To/Subject	"Gurpartap Singh" <gurpartap@patriots.in> /

CONCLUSIONS

The SMTP capture for Lab 4 was fully reconstructed in plaintext. The session originated from 10.10.1.4:1470 (Microsoft Office Outlook 12.0, MAC 00:0e:0c:1c:3c:17:c2) to 74.53.140.153:25 (xc90.websitewelcome.com, Exim 4.69, MAC 00:1f:33:d9:81:60). The client authenticated via AUTH LOGIN, transmitting Base64-encoded credentials that were decoded offline in a controlled environment. An email from gurpartap@patriots.in to raj_deol2002in@yahoo.co.in with subject SMTP, dated Mon, 5 Oct 2009 11:36:07 +0530, was recovered. The message is multipart/mixed containing a multipart/alternative body with both plain-text and HTML versions; no attachment is present.

Forensically, the exercise demonstrates three key lessons:

- (1) Base64 is not encryption and provides no confidentiality.
- (2) Port numbers alone are insufficient to determine whether encryption is present protocol negotiation must be examined.
- (3) Packet-level capture combined with stream reassembly is a powerful and complete evidence-recovery technique when encryption is absent. The capture also illustrates practical investigative realities out-of-order packets, ICMP fragmentation events, and mixed transport modes that any real forensic workflow must accommodate.